00c – Faden-Karte: Der Weg der Template-ID

Zweck: Den Faden am Sessionanfang in 5 Minuten wiederfinden. Diese Seite ändert sich selten. Sie zeigt **immer dieselbe Geschichte**: Wie die Nummer des Bauplans (Template-ID) durch die Pipeline wandert. Seit 16.10. kommt **Teil D** dazu: Phase 3 – wo der State liegt und was `terraform init` tut. Seit 18.10. **Teil E**: der Weg der Schlüssel (Tresor → Schlüssel → Tür).

Was auf dieser Karte steht – und was nicht:

Hier (Faden-Karte)	Nicht hier
Wie die Pipeline **dauerhaft** funktioniert – was bei **jedem** Lauf gilt	**Einmalige** Schritte: Umzug, Aufräumen, PR vorbereiten und prüfen
Beispiel: was `terraform init` tut, wo der State liegt	Beispiel: alte `terraform.tfstate` vor dem PR wegräumen
→ zum Lernen und Wiederholen	→ steht in `01_aktueller_stand.md` (Plan, offene Punkte) und in der **Session-Zusammenfassung** des Tages (Ablauf)

So benutzt du sie am Sessionanfang:

1. Teil A (Landkarte) kurz anschauen – wo bin ich?
2. Teil B von oben nach unten lesen, **jede Zeile von rechts nach links**.
3. Teil C: Claude stellt **eine** Einstiegsfrage, du antwortest mit der Karte daneben.
4. Erst danach: Gedächtnis-Refresh zu dem, was neu ist.

Grundfrage bei jedem Namen: Bauplan oder Haus? Steht `template` im Namen → **Bauplan** (Template, z. B. 100 / 2000). Ein **Haus** ist eine VM (z. B. 1001 `web_server`, 100 `test_vm`).

Begriffe (seit 10.10.): Nur noch **State** bzw. die Datei `terraform.tfstate` – nicht mehr „Grundbuch“.

Zwei Arten von Nummern auf dieser Seite:

Nummer	Wo	Bedeutung
① – ⑤ (im Kreis)	in den **Überschriften** von Teil B	die **fünf Stationen** der 2000: an welchem Ort sie gerade ist und was dort mit ihr passiert
1, 2, 3 … und **a, b, c …**	**unter den Klammern** einer Codezeile	die **Teile dieser einen Zeile**; die Tabelle darunter sagt, wer den Teil bestimmt und was er bedeutet

Überschriften **ohne** Kreis-Nummer sind **Zwischenschritte**: Dort passiert etwas, das die 2000 weiterbringt, aber sie wird dort nicht abgelegt.

Zeilennummern: `pipeline.yml`, Stand 10.10.2026, **lokale Fassung** auf `soar-dev-server` (mit noch nicht gepushten Kommentaren, geparkt für den Kommentar-PR). **Auf GitHub** (damit läuft die Pipeline heute) liegt alles ab Job 3 Step 2 um **+1 bis +2** Zeilen weiter unten: z. B. `apply` 341, ④ 361, ⑤ 382. Nach dem Kommentar-PR gelten die Nummern auf dieser Karte.

Teil A – Die Landkarte (wo bin ich?)

Job	Was er tut	Braucht Werte von	Gibt Werte weiter an
JOB 0 `detect-changes`	schaut, welche Dateien geändert wurden	–	Job 1 (`packer`)
JOB 1 `build-template`	**Packer** baut den Bauplan	Job 0	Job 3 (`new_template_id`)
JOB 2 `security-scan`	Checkov prüft Terraform-Code	–	–
JOB 3 `deploy-vm`	**Terraform** baut die Häuser	Job 1	Job 5 (IPs), Job 6 (`effective_template_id`)
JOB 4 `ansible-security-scan`	Checkov prüft Ansible-Code	–	–
JOB 5 `configure-vm`	**Ansible** richtet die Häuser ein	Job 3	–
JOB 6 `promote-template`	gibt den Bauplan frei	Job 3	–

Die Geschichte unten spielt in **Job 3** und endet in **Job 6**.

Teil B – Die Kette, Zeile für Zeile (Fall 2: Packer lief nicht → 2000)

Wer handelt? Du schreibst den Code (Anweisungen). Terraform, Bash und GitHub führen ihn beim Lauf aus.

Die fünf Stationen auf einen Blick

Station	Ort	Was die Station verkörpert	Wer handelt
①	`variables.tf`	**Herkunft**: Hier steht die 2000 als Ersatzwert	du (einmal geschrieben)
②	`outputs.tf`	**Anweisung**: „Schreib die 2000 nach dem `apply` in den State“	du (einmal geschrieben)
③	State (`terraform.tfstate`)	**Ablage**: Hier liegt die 2000 nach dem `apply`	Terraform schreibt (Step 6, Z. 339)
④	Job 3 Step 7, Z. 359	**Abholen**: Die 2000 wird aus dem State gelesen	Terraform liest, Bash legt ab
⑤	Job 3 Step 7, Z. 380	**Weitergeben**: Die 2000 geht in die Übergabe-Datei	Bash

Danach: Schaufenster im Job-Kopf (Z. 256) → Job 6.

① Station 1 – Herkunft: `variables.tf`

Hier steht die 2000 als Ersatzwert. (vereinfacht in einer Zeile, in der Datei mehrzeilig)

```
variable "test_template_id"   { default = 2000 }
└──────┘ └─────────────────┘   └─────┘   └──┘
   1             2              3       4
```

Nr	Teil	Wer bestimmt das?	Bedeutung
1	`variable`	festes Wort von **Terraform**	„Ich melde eine Variable an“
2	`"test_template_id"`	**du**, frei gewählt	Name der Variable
3	`default`	festes Wort von **Terraform**	Ersatzwert, wenn kein Zettel da ist
4	`2000`	**du**	der Ersatzwert selbst

Zwischenschritt – Job 3 Step 6, Z. 336/337: Zettel hinlegen (oder nicht)

Entscheidet, ob Terraform die 2000 nimmt oder einen anderen Wert.

```
export TF_VAR_test_template_id="${{ steps.effective-template.outputs.template_id }}"
└────┘ └──┘└──────────────┘↑ └──────────────────────────────────────────────────────┘
  1      2         3       4                        5
```

Nr	Teil	Wer bestimmt das?	Bedeutung
1	`export`	festes Wort von **Bash**	„Leg einen Zettel in die Umgebung“
2	`TF_VAR_`	fester Anfang, von **Terraform** vorgegeben	„Dieser Zettel ist für Terraform“
3	`test_template_id`	**du**, muss exakt so heißen wie in `variables.tf`	für welche Variable der Zettel ist
4	`=`	**Bash**	Zuweisung, ohne Leerzeichen davor oder danach
5	`${{ … }}`	**GitHub** setzt es **vor** dem Start ein	der Wert: Auftrag aus Step 2

Nr. 5 genauer zerlegt:

```
${{ steps.effective-template.outputs.template_id }}
    └──┘ └──────────────┘ └────┘ └──────────┘
     a          b          c         d
```

Nr	Teil	Wer bestimmt das?	Bedeutung
a	`steps`	festes Wort von **GitHub**	„ein Step in **diesem** Job“
b	`effective-template`	**du**: die `id:` von Step 2 (Z. 274)	welcher Step
c	`outputs`	festes Wort von **GitHub**	„dessen Übergabe-Datei“
d	`template_id`	**du**: der Name links vom `=` in Z. 284	welcher Eintrag darin

Fall 2: Der Auftrag ist leer → `if` in Z. 336 ist falsch → **Z. 337 wird übersprungen** → **kein Zettel** → Terraform nimmt die 2000 aus ①.

Zwischenschritt – Job 3 Step 6, Z. 339: Terraform baut und schreibt

Hier wird die 2000 benutzt (Klonen) und nach Anweisung ② in den State ③ geschrieben.

```
terraform apply -auto-approve
└──────┘ └───┘ └──────────┘
    1       2         3
```

Nr	Teil	Wer bestimmt das?	Bedeutung
1	`terraform`	das **Programm** auf dem Runner	Terraform wird gestartet
2	`apply`	festes Wort von **Terraform**	„Bauen und danach in den State schreiben“
3	`-auto-approve`	Schalter von **Terraform**	ohne Rückfrage „Wirklich?“

Was Terraform bei `apply` tut, der Reihe nach:

1. liest den **Code** (alle `.tf`-Dateien) → was es geben **soll**
2. setzt den **Lock** auf den State
3. liest den **State** → was es **weiß**
4. fragt Proxmox: Stehen die VMs aus dem State noch?
5. vergleicht Code ↔ State → was fehlt?
6. lässt Proxmox bauen; Proxmox meldet zurück (Nummer, …)
7. **schreibt in den State**, erst **nachdem** die VM steht (Quittung)
8. entfernt den Lock

② Station 2 – Anweisung: `outputs.tf`

Hier steht, dass die 2000 nach dem `apply` in den State soll. (Du hast das vorher geschrieben; ausgeführt wird es bei Z. 339.)

```
output "used_test_template_id"   { value = var.test_template_id }
└────┘ └──────────┬──────────┘    └───┘   └─┘ └──────┬──────┘
  1               2                3      4         5
```

Nr	Teil	Wer bestimmt das?	Bedeutung
1	`output`	festes Wort von **Terraform**	„Nach dem `apply` unter diesem Namen in den State schreiben“
2	`"used_test_template_id"`	**du**, frei gewählt	Name des Eintrags im State
3	`value`	festes Wort von **Terraform**	welcher Wert eingetragen wird
4	`var`	festes Wort von **Terraform**	„aus einer Variable“
5	`test_template_id`	**du**: Name aus `variables.tf` (①)	aus welcher Variable

Du schreibst diese Zeile einmal. **Terraform** führt sie bei jedem `apply` aus.

③ Station 3 – Ablage: der State (`terraform.tfstate`)

Hier liegt die 2000 nach dem `apply`.

Name	Wert
`used_test_template_id`	`2000`

(Am 09.10. im echten State auf `soar-dev-server` per `grep` gesehen.) Seit Phase 3 liegt der State **bei HCP** statt im Runner-Ordner → Teil D.

④ Station 4 – Abholen: Job 3 Step 7, Z. 359

Hier wird die 2000 aus dem State gelesen.

```
USED_TEMPLATE_ID=$(terraform output -raw used_test_template_id 2>/dev/null)
└──────────────┘└┘└───────┘ └────┘ └┘ └──────────────────────┘ └─────────┘
       1         2     3       4    5            6                7
```

Nr	Teil	Wer bestimmt das?	Bedeutung
1	`USED_TEMPLATE_ID`	**du**, frei gewählt (GROSS)	Bash-Variable, **entsteht hier**
2	`=$( … )`	**Bash**	„Führ den Befehl in der Klammer aus, leg das Ergebnis links ab“
3	`terraform`	das **Programm**	Terraform wird (zum zweiten Mal) gestartet
4	`output`	festes Wort von **Terraform**	„Lies einen Eintrag aus dem State“ – liest **nur** den State
5	`-raw`	Schalter von **Terraform**	nur der nackte Wert, ohne Anführungszeichen
6	`used_test_template_id`	**du**: Name aus `outputs.tf` (②, klein)	welcher Eintrag
7	`2>/dev/null`	**Bash**	Fehlermeldungen (Kanal 2) in die Mülltonne

Zwei fast gleiche Namen: **1 GROSS** = Bash-Variable, **6 klein** = Name im State.

⑤ Station 5 – Weitergeben: Job 3 Step 7, Z. 380

Hier geht die 2000 in die Übergabe-Datei.

```
echo  "used_template_id=$USED_TEMPLATE_ID"  >> $GITHUB_OUTPUT
└──┘  └──────────┬──────┘└──────────┬──────┘  ↑  └─────┬──────┘
 1          2        3        4          5         6
```

Nr	Teil	Wer bestimmt das?	Bedeutung
1	`echo`	festes Wort von **Bash**	„Gib aus“
2	`used_template_id`	**du**, frei gewählt	Name für die Übergabe
3	`=`	Format von **GitHub**	trennt Name und Wert
4	`$USED_TEMPLATE_ID`	**Bash** setzt den Wert ein	2000 aus ④
5	`>>`	**Bash**	an eine Datei **anhängen**
6	`$GITHUB_OUTPUT`	**GitHub** gibt den Dateipfad vor	die Übergabe-Datei dieses Steps

(In dieser Darstellung stehen zur besseren Lesbarkeit zwei Leerzeichen nach `echo`; in der Datei ist es eins.)

Zwischenschritt – Job 3 Job-Kopf, Z. 256: ins Schaufenster stellen

Macht die 2000 aus ⑤ für Job 6 sichtbar.

```
effective_template_id: ${{ steps.get-ip.outputs.used_template_id }}
└──────────┬─────────┘↑ └┬┘ └──┬──┘ └──┬──┘ └──┬──┘ └──────┬──────┘
           1          2  3     4      5       6           7
```

Nr	Teil	Wer bestimmt das?	Bedeutung
1	`effective_template_id`	**du**, frei gewählt	Name im Schaufenster des Jobs
2	`:`	**YAML**	trennt Name und Wert
3	`${{` … `}}`	**GitHub**	wird am **Job-Ende** eingesetzt
4	`steps`	festes Wort von **GitHub**	„ein Step in diesem Job“
5	`get-ip`	**du**: die `id:` von Step 7 (Z. 351)	welcher Step
6	`outputs`	festes Wort von **GitHub**	„dessen Übergabe-Datei“
7	`used_template_id`	**du**: Name aus ⑤ Nr. 2	welcher Eintrag

Job 6 liest es mit `needs.deploy-vm.outputs.effective_template_id` (Z. 616, 625, 629).

Die ganze Kette in einer Zeile

```
① variables.tf (2000) → kein Zettel → apply → ② Anweisung → ③ State
  → ④ abholen → ⑤ weitergeben → Schaufenster → Job 6
```

Teil C – Einstiegsfragen (Claude stellt EINE davon)

1. Lies die Zeile von ④ von rechts nach links: **Wer tut was?**
2. Wer **schreibt** die 2000 in den State (③), und bei welchem Befehl?
3. Warum gibt es in Fall 2 keinen Zettel? (Welche Zeile wird übersprungen?)
4. `outputs.tf` (②): Welche Teile hast **du** gewählt, welche sind feste Wörter?
5. Bauplan oder Haus: 100, 2000, 1001 – und wer macht sie?
6. (neu 10.10.) Was tut Terraform bei `apply` **vor** und **nach** dem Bauen mit dem State?
7. (neu 10.10.) Nenne zu jeder Station ①–⑤ in einem Wort, was sie verkörpert.
8. (neu 16.10.) Was holt `terraform init`, und **wo** legt es das ab?
9. (neu 16.10.) Code, Proxmox, State, Provider – wer ist Soll, Ist, Notiz, Übersetzer?
10. (neu 16.10.) Ist HCP ein Provider? Wenn nicht – was dann?
11. (neu 18.10.) Tresor, Schlüssel, Tür: Was ist GitHub Secrets, was ist `TF_API_TOKEN`, was ist HCP?
12. (neu 18.10.) Mit welchem HCP-Token meldet sich Terraform an – in der **Pipeline** und im **Terminal**? Wo liegt er jeweils?

Teil D – Phase 3: Wo liegt der State, und was tut `init`? (seit 16.10.)

D1 – Die vier Teile

Teil	Rolle	Wo
Code (`.tf`-Dateien)	**Soll** – was es geben soll	Repo, Ordner `terraform/`
Proxmox	**Ist** – was wirklich steht	Minisforum im Heimnetz
State	**Notiz** – was Terraform weiß	**bei HCP**, Workspace `02-CI-CO-VM`
Provider `bpg/proxmox`	**Übersetzer** Terraform ↔ Proxmox	`terraform/.terraform/providers/`

```
Terraform → Plugin bpg/proxmox → Proxmox-API (Port 8006)
```

D2 – Drei Zutaten, damit Terraform den State bei HCP findet

Zutat	Wo	Wofür
Adresse	`provider.tf`, `cloud`-Block (Z. 41–47): Organisation `randolph-iacdeveloping`, Workspace `02-CI-CO-VM`	**wohin**
Schlüssel	Pipeline: Secret `TF_API_TOKEN` → Job 3 Job-Kopf `env: TF_TOKEN_app_terraform_io`; Terminal: `terraform login`	**darf rein**
Ablage	HCP-Workspace, Execution Mode **Local**	**wo** die Notiz liegt

Local = Terraform läuft weiter auf dem Runner (redet mit Proxmox im Heimnetz), HCP hält nur **State + Lock**. Verbindung: Terraform → HTTPS → `app.terraform.io`, **nur von innen nach außen**, kein offener Port zu Hause.

D3 – `terraform init` (Job 3, auf dem Branch Z. 311)

```
terraform init -input=false
└──┬───┘ └┬┘ └─────┬─────┘
   1      2        3
```

Nr	Teil	Wer bestimmt das?	Bedeutung
1	`terraform`	das **Programm** auf dem Runner	Terraform wird gestartet
2	`init`	festes Wort von **Terraform**	„Ordner vorbereiten: Plugin holen, State-Adresse einrichten“
3	`-input=false`	Schalter von **Terraform**	keine Rückfragen – kommt eine, **bricht** `init` ab

Was `init` tut, der Reihe nach:

#	Terraform …	mit wem
1	liest `provider.tf`, findet den `cloud`-Block (Adresse)	lokal
2	sucht den Schlüssel (`TF_TOKEN_app_terraform_io` bzw. `terraform login`)	lokal
3	meldet sich bei `app.terraform.io` an, prüft Organisation + Workspace	HCP
4	lädt das **Plugin** `bpg/proxmox` (oder nimmt es aus dem Zwischenspeicher)	Terraform Registry
5	merkt sich die Adresse in `.terraform/`	lokal

D4 – Was liegt wo im Ordner `terraform/`?

```
terraform/
├── provider.tf           ← du: cloud-Block (Adresse) + required_providers
├── .terraform.lock.hcl   ← init: genaue Plugin-Version
└── .terraform/           ← init legt den Ordner an
    ├── providers/        ← das Plugin bpg/proxmox (ein eigenes Programm)
    └── terraform.tfstate ← NUR die Adressnotiz „State liegt bei HCP“
                            – NICHT der State
```

Jeder Ordner, in dem Terraform laufen soll, braucht **einmal** `init`.

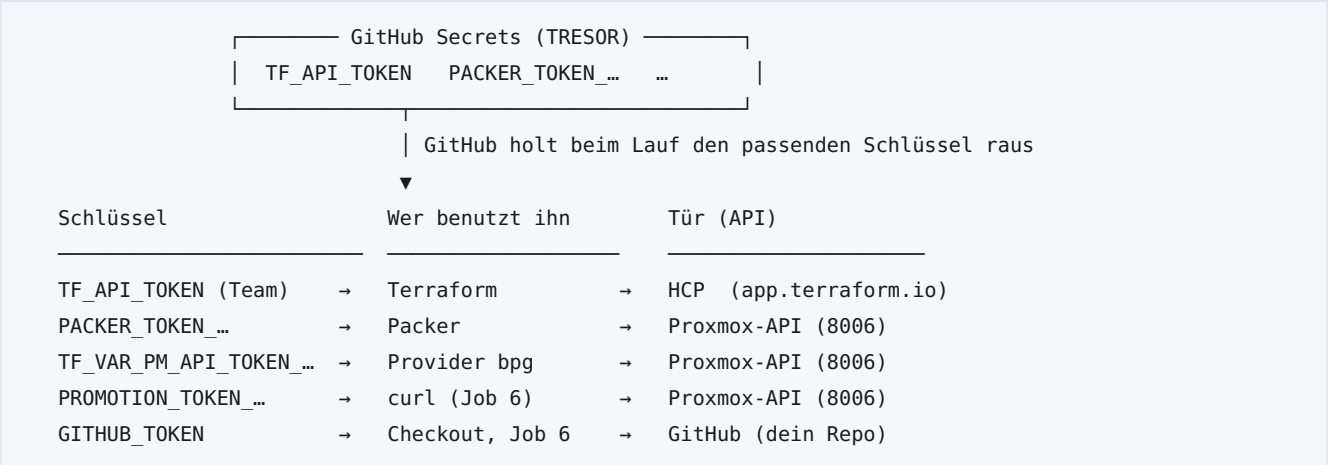
D5 – Typische Verwechslungen

Falsch	Richtig
HCP ist der Provider	HCP = **Ablage** (`cloud`-Block, *wo* notiert wird); Provider = Plugin, *mit wem* gebaut wird
`init` holt die Proxmox-API	`init` holt das **Plugin**, das **mit** der API spricht; die API bleibt auf dem Proxmox-Server
Das Plugin steckt in Terraform	Es liegt **neben** dem Projekt in `.terraform/providers/`
`.terraform/terraform.tfstate` ist der State	nur die Adressnotiz; der State liegt bei HCP
Die `.gitignore` ist ein Ablageort	sie hält Dateien nur **aus Git heraus**

Teil E – Der Weg der Schlüssel: Tresor → Schlüssel → Tür (seit 18.10.)

Grundbild: Jede API ist eine **Tür**, jede Tür braucht ihren **eigenen Schlüssel** (Token). HCP ist auch nur eine API (`app.terraform.io`), genau wie Proxmox (Port 8006). **GitHub Secrets ist kein Schlüssel, sondern der Tresor.**

E1 – Pipeline: wer bekommt welchen Schlüssel für welche Tür?



`GITHUB_TOKEN` liegt **nicht** im Tresor: GitHub erzeugt ihn bei jedem Lauf automatisch und nimmt ihn danach wieder weg („Post Repository auschecken“).

Dazu ein Schlüssel**paar** für SSH (keine API, aber auch eine Tür): `SSH_PUBLIC_KEY` → Job 3 baut das **Schloss** in die VM ein · `SSH_PRIVATE_KEY` → Job 5, **Ansible** öffnet damit die VM.

Drei verschiedene Proxmox-Tokens (Packer, Terraform, Job 6) = jeder Job hat seine **eigene Keycard** → Prinzip **minimale Rechte**.

E2 – Zwei Arten, einen Schlüssel an Terraform zu geben

Anfang	Wer liest ihn	Wohin geht der Wert
`TF_TOKEN_…`	**Terraform selbst** (Programm)	Anmeldung bei HCP – **nicht** in deinen Code
`TF_VAR_…`	**dein Code**	wird zu `var.…` in `variables.tf`, z. B. Keycard für den Provider

Ebene in `pipeline.yml`	Einrückung	gilt für
`env:` im **Job-Kopf** (z. B. `TF_TOKEN_app_terraform_io`)	6 Leerzeichen	**alle Steps** des Jobs
`env:` in einem **Step** (z. B. `TF_VAR_pm_api_url`)	10 Leerzeichen	**nur diesen Step**

Ablauf: ① **GitHub** holt vor dem Job-Start den Wert aus dem Tresor und legt ihn als Umgebungsvariable hin. ② **Terraform** liest die Adresse im `cloud`-Block und sucht **gezielt** den passenden Schlüssel – es kennt den Tresor-Namen `TF_API_TOKEN` gar nicht.

E3 – Welcher HCP-Token wann?

Situation	HCP-Token	Art in HCP	Wo liegt er	Wie findet Terraform ihn
Pipeline (Job 3)	`pipeline-02-CI-CO-VM`	**Team-Token**	GitHub Secret `TF_API_TOKEN`	Umgebungsvariable `TF_TOKEN_app_terraform_io`
Terminal (z. B. `~/umzug-02`)	`soar-dev-server-uebung`	**User-Token**	`~/.terraform.d/credentials.tfrc.json`	gespeichert von `terraform login`

Terraform meldet sich bei **jedem** Befehl an, der den State braucht: `init`, `plan`, `apply`, `state pull`, `state list`, `output`. Reihenfolge: zuerst `TF_TOKEN_app_terraform_io`, sonst die Datei.

Röntgen-Befehl für jede Pipeline: `grep -n "secrets\." pipeline.yml` zeigt alle Stellen, an denen ein Schlüssel aus dem Tresor eingesetzt wird (nur Namen, nie Werte).

Pflege: Nur ändern, wenn sich die Pipeline an diesen Stellen ändert (z. B. nach der geplanten Umbenennung von `effective_template_id`) oder ein grundlegend neuer Faden dazukommt. Regel (16.10.): In die Faden-Karte kommt nur, was dauerhaft bei jedem Lauf gilt. Einmalige Schritte (Umzug, Aufräumen, PR vorbereiten/prüfen) gehören in `01` und die Session-Zusammenfassung. Letzte Änderung: 18.10.2026 – Teil E (Weg der Schlüssel: Tresor/Schlüssel/Tür, `TF_TOKEN_` vs. `TF_VAR_`, Job-Kopf vs. Step, Team- vs. User-Token), Einstiegsfragen 11–12. Davor 16.10.2026 – Teil D (Phase 3: vier Teile, drei Zutaten, `init`, Ordner `terraform/`, Verwechslungen), Einstiegsfragen 8–10. Davor 10.10.2026: Klammern mit Nummern + Tabellen, Stationen ①–⑤, „Grundbuch“ → State.